Introduction to Data Visualization

Data Visualization Using Python

- In today's world, a lot of data is being generated on a daily basis. And sometimes to analyze this data for certain trends, patterns may become difficult if the data is in its raw format. To overcome this data visualization comes into play. Data visualization provides a good, organized pictorial representation of the data which makes it easier to understand, observe, analyze. In this tutorial, we will discuss how to visualize data using Python.

Libraries:

- Matplotlib
- Seaborn
- Bokeh
- Plotly

Why Python?

- It is very well known for data analysis and visualizations because of the vast libraries it provides such as Pandas, Numpy, Matplotlib, and others.

Installation of Pandas

- If you have Python and PIP already installed on a system, then installation of Pandas is very easy.
- Install it using this command:

```
C:\Users\Your Name>pip install pandas
```

Import Pandas

- Once Pandas is installed, import it in your applications by adding the import keyword:

```
import pandas
```

Example 1

```
import pandas

mydataset = {
  ‘animals': [“Cat", “Dog", “Rabbit"],
  ‘height’: [13, 17, 8]
}

myvar = pandas.DataFrame(mydataset)

print(myvar)
```

Pandas Series

- A Pandas Series is like a column in a table.

- It is a one-dimensional array holding data of any type.

Example 2

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a)

print(myvar)
```

Labels

- If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

- This label can be used to access a specified value.

```
print(myvar[0])
```

Create Labels

- With the index argument, you can name your own labels

```
import pandas as pd

a = [1, 7, 2]

myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```

Create Labels

- When you have created labels, you can access an item by referring to the label.

```
print(myvar["y"])
```

Key/Value Objects as Series

- Create a simple Pandas Series from a dictionary:

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(calories)

print(myvar)
```

DataFrames

- Data sets in Pandas are usually multi-dimensional tables, called DataFrames.

- Series is like a column, a DataFrame is the whole table.

Example 3

- Create a DataFrame from two Series:

```
import pandas as pd

data = {
  “age": [42, 38, 39],
  “weight": [50, 40, 45]
}

myvar = pd.DataFrame(data)

print(myvar)
```

Locate Row

- Pandas use the loc attribute to return one or more specified row(s)

```
#refer to the row index:
print(df.loc[0])
```

- Return row 0 and 1:

```
#use a list of indexes:
print(df.loc[[0, 1]])
```

Named Index

- Add a list of names to give each row a name:

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
```

Locate Named Index

```
#refer to the named index:
print(df.loc["day2"])
```

Load Files Into a DataFrame

- If your data sets are stored in a file, Pandas can load them into a DataFrame.

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df)
```

Viewing the Data

- The head() method returns the headers and a specified number of rows, starting from the top.

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
```

Viewing the Data

- There is also a tail() method for viewing the last rows of the DataFrame.

- The tail() method returns the headers and a specified number of rows, starting from the bottom.

Information about the Data

- The DataFrames object has a method called info(), that gives you more information about the data set.

```
print(df.info())
```

Removing Duplicates

```
print(df.duplicated())

df.drop_duplicates(inplace = True)
```

Tabulate

```
from tabulate import tabulate
```

<statement...>

```
print(tabulate(df, headers='keys', tablefmt='psql')
```

Methods to Understand Data Visualization

- Storing DataFrame in CSV Format

Pandas provide to csv(‘filename’, index = “False|True”)a function to write DataFrame into a CSV file. Here *filename* is the name of the CSV file that you want to create and *index* tells the index (if Default) of DataFrame should be overwritten or not. If we set index = False then the index is not overwritten. By Default value of index is TRUE then the index is overwritten.